

1. Introduction

This project is a privacy-focused tool designed to help individuals develop deeper self-awareness of their emotions and mental states. Users engage with an emotional stimulus, a short video of carefully chosen images, while their facial expressions are analysed in real time. Their verbal responses are then transcribed and analysed for emotional tone. The system synthesises this data to generate a personalised psychological profile, identify emotional patterns, and provide targeted recommendations for stress management, self-improvement, and mental wellness. All processing occurs locally on the user's device with zero data transmission. This is an introductory tool for people who lack access to therapy or prefer an impersonal first step. It is not a replacement for professional mental health care.

The project is based on Template 4.1: Orchestrating AI models to achieve a goal.

Most people struggle to understand their own emotions. Professional support is expensive and not always accessible. Mental health disorders constitute five of the top ten causes of disability across Europe, with an estimated total cost exceeding €600 billion annually (Laidi et al., 2023). This tool targets two groups: those who cannot afford professional therapy, and those who experience anxiety or stigma when discussing mental health with strangers.

Similar technology already exists but serves different purposes. Affectiva, a commercial product born out of MIT Media Lab, uses computer vision to detect emotions from facial expressions in real time. However it is built for corporate use, its primary application is measuring consumer reactions to advertisements. The insights go to the advertiser, not the person being analysed. It is expensive, closed source, and inaccessible to individuals.

A more recent example is a 2025 academic paper from Brunel University London, which combined facial emotion recognition, speech-to-text, and sentiment analysis, feeding everything into a language model, a pipeline technically very similar to this project. However their system only processes pre-recorded video, is not interactive, and uses GPT-4o, a paid cloud service. The authors themselves flagged this as a privacy concern.

This project addresses both gaps: it uses a live webcam feed instead of pre-recorded video, and runs the language model entirely locally using Ollama. Rather than producing a static report for a researcher or feeding data to an advertiser, the system opens an interactive conversation with the user, putting the insight back in the hands of the person it came from.

2. Literature Review

Examples of similar projects

Abby AI, Medical Decision Support System

Abby AI is an Australian medical tool that uses AI to prepare patient profiles before clinical appointments. It collects medical history, past test results, and medication records, synthesises them, and presents a structured summary to the doctor before they see the patient (Abby Health, 2026). The doctor remains in control at all times, Abby does not diagnose or prescribe.

Both systems collect information about a person and generate a profile from it. The difference is who that profile is for. Abby serves the clinician, the insight goes to the professional, not the patient. This project does the opposite: there is no doctor, no intermediary. The profile goes directly to the user.

Affectiva, Emotion AI Software

Miller (2017) describes Affectiva as a company that makes AI software to detect human emotions, it started from MIT Media Lab. The company analyzes facial expressions, voice tone, and body movements to understand what people feel. And according to an article from Affectiva themselves (Affectiva, 2016) they used the Facial Action Coding System (FACS), developed by Paul Ekman, to label facial expressions.

Affectiva uses cameras and microphones to detect emotions in real-time. It analyzes facial expressions, voice tone, and body language. The company has studied over 10 million face videos from 90 countries. It can detect emotions like happiness, anger, confusion, and frustration.

My project aims at using similar technology, also based on Ekman's work, to detect emotions in real time from a live feed, and give immediate feedback to the user, for free.

Affectiva is used by large companies for different purposes. Advertising companies use it to measure how people react to ads. Car companies use it to monitor drivers and detect if they are tired or distracted. Companies use it to understand customer emotions in call centers. Gaming and education companies use it to detect if users are bored.

Both Affectiva and my project use AI to detect emotions. But it is a paid software for big corporations to make more money and understand customers. My project is a free software for people to understand their own emotions better.

Techniques and methods

For my project I combine several pre-trained models to achieve a common goal. The domains covered are four: vision, audio, text, and language generation. The library and model choices below were explored starting from more demanding options and downgraded to lighter frameworks where hardware constraints required it.

OpenCV, Computer Vision Library

OpenCV is an open-source library for detecting faces and tracking movement in videos. It runs locally on the user's machine, so the data stays private. The library is cross-platform and licensed as free and open-source software (OpenCV, 2026), making it well suited to this project.

Whisper / faster-whisper, Speech-to-Text Model

For speech recognition the project uses Whisper, an open-source speech recognition model created by OpenAI (Radford et al., 2022), trained on 680,000 hours of multilingual audio and supporting 99 languages. It runs locally without needing an internet connection. In practice the project uses faster-whisper, a reimplementation that runs roughly four times faster on CPU at equivalent accuracy, which matters given the target hardware has no dedicated GPU.

Whisper transcribes what the user says during the emotional assessment; the transcribed text is then analysed for sentiment and emotional tone (Radford et al., 2022).

faster-whisper was benchmarked directly against the original openai-whisper on the same audio file (both at the tiny model size, see Evaluation, Stage 2 results). faster-whisper was confirmed faster on transcription and, on this test file, more accurate: openai-whisper mis-transcribed one word ('microphone' as 'Micah Cup') where faster-whisper transcribed the file correctly. Vosk was also considered as a lighter-weight fallback option (see Design chapter) but was not directly benchmarked, since the openai-whisper comparison already demonstrates the accuracy/performance case for faster-whisper against the model it is built on.

Sentiment / Text Emotion Detection

To detect emotion in text, the transcribed speech from Whisper is fed to a pre-trained model to identify emotional tone and sentiment. These models can be downloaded and run locally, using deep learning to classify text as positive, negative, or neutral, or to detect more nuanced emotions like frustration, joy, or sadness (Hugging Face, 2024).

Three candidates were benchmarked on a fixed set of test transcripts: VADER, a rule-based analyser (Hutto & Gilbert, 2014); tabularisai/multilingual-sentiment-analysis; and j-hartmann/emotion-english-distilroberta-base. j-hartmann was selected, both for its higher accuracy on the test set and because it shares the same seven-emotion label space as the DeepFace facial-emotion model, allowing facial and verbal emotion to be compared directly. Full results are given in the Evaluation chapter.

Ollama, Local Language Model Framework

For the interactive component the project uses Ollama, a framework that allows large language models to run locally without an internet connection or cloud services (Ollama, 2024). Ollama generates the final psychological profile and personalised recommendations from the combined facial emotion, transcribed speech, and sentiment data. Because Ollama

runs entirely on the user's machine, all emotional data stays private and is never sent to external servers (Ollama, 2024).

llama3.2 (approximately 2GB) was used for initial integration, chosen as the smallest widely-used Ollama model and confirmed to run on CPU-only consumer hardware without a dedicated GPU.

llama3.2 and qwen2.5:3b were benchmarked on an identical sample prompt (see Evaluation, Table 3). qwen2.5:3b was notably faster on both time-to-first-token and total generation time, but its reply did not include a recommendation to seek professional help despite the sample containing a fear signal comparable to earlier tests where llama3.2 reliably included one. Given the project's severity-assessment design requirement, llama3.2 was retained as the primary model, with qwen2.5:3b noted as a faster alternative worth further, more systematic safety testing.

Study cases and research evidence

To assess the effectiveness of AI in psychological diagnosis this project draws on a study by Alyssa Ezell, conducted at Widener University, testing whether six AI chatbots - three general-use (ChatGPT, Gemini, Claude) and three therapist-specific (Pi, Panda, Abby) - could accurately diagnose mental health conditions and create treatment plans (Ezell, 2026).

General-use chatbots provided diagnoses in 100% of cases, but only 36.7% were completely correct; in 40% of cases, chatbots gave both correct and incorrect diagnoses simultaneously (Ezell, 2026). Therapist-specific chatbots were more cautious, providing diagnoses only 53.3% of the time (Ezell, 2026). For treatment planning, general-use chatbots recommended plans in 93.3% of cases, most commonly Cognitive Behavioral Therapy, psychiatric medication, and social support, but these recommendations were very generic (Ezell, 2026).

Ezell's research demonstrates that AI can provide mental health information, but current systems struggle with accuracy, personalisation, and transparency. This project aims to narrow those gaps through real-time emotion detection, individual focus, and privacy-preserving local processing.

3. Design

Domain

Laidi et al. (2023) report that mental disorders constitute five of the top ten causes of disability across Europe, with an estimated total cost exceeding 4% of GDP, more than €600 billion annually. They also note that these conditions often begin early in life, with significant impacts on work productivity and quality of life.

The primary users of this product are individuals aged 18-45 seeking mental health support in Europe. The scale of the problem is considerable: in France alone, Laidi et al.

(2023) estimate the annual cost of mental health at €163 billion in 2018, broken down into direct healthcare costs (14%), productivity loss (27%), and loss of quality of life (51%). These costs reflect both the prevalence of mental health conditions and barriers to access, with anxiety and depressive disorders being the most common conditions, affecting millions of people across the continent.

Affordability is not the only issue: discomfort with human interaction is also a critical barrier. While people generally prefer human doctors to AI systems, recent evidence shows that interaction with a fully automated AI chatbot can be an effective approach in the diagnosis and treatment of psychiatric disorders (Lee et al., 2022); some individuals may engage with AI-based tools when human interaction feels inaccessible or uncomfortable (Riedl et al., 2024).

This tool targets two user groups: those who cannot afford professional therapy, and those who experience anxiety or stigma when discussing mental health with strangers. The system is designed to assess the severity of a case; if a user's profile indicates serious mental health concerns, the system actively encourages them to seek professional help. It is not a replacement for therapy, but a meaningful first step toward self-awareness and care-seeking.

Design justification

The design choices in this project are based on the specific needs identified in the domain and user analysis: privacy, affordability, accessibility, and responsible care-seeking.

Fully local processing

To address privacy concerns that are especially important in mental health contexts, the entire system runs locally, with no internet connection or cloud services. Riedl et al. (2024) found that perceived privacy invasion was higher in AI-based interactions, and higher still in psychiatry than in other medical fields, where patients fear data breaches when discussing personal conditions. By keeping all emotional data on the user's machine, the system removes this barrier entirely. This is also what separates the project from commercial products like Affectiva, which send insights to third parties rather than the user.

Multiple models across different domains

The system combines facial emotion recognition, speech transcription, sentiment analysis, and a local language model. This is because people often find it hard to put their feelings into words, and what they say does not always match what their face shows. Using more than one source of information gives a fuller picture than any single model could. Recent work in the field follows the same logic: the Brunel University study (2025), for example, combined facial emotion recognition with text sentiment because no single model captured the full emotional picture on its own.

Interactive conversation

After analysing the user's emotions, the system opens a dialogue through the local language model instead of producing a static diagnosis. This matters because the target users are often approaching mental health for the first time and need guidance, not just a label. An interactive format lets them ask questions, explore recommendations, and feel heard. Lee et al. (2022) found that AI chatbots can foster rapport and self-disclosure, which is valuable for users who feel anxious or embarrassed talking to a stranger.

Severity assessment

The system is not designed to replace therapy. When a user's profile suggests serious concerns, it encourages them to seek professional help. The evidence is clear that people prefer and benefit from human care (Riedl et al., 2024), and an AI tool that masks the need for real treatment would be harmful. This project recognises the importance of human connection in effective mental healthcare, and treats itself as a first step rather than a substitute.

Inclusive design

Structure of the project

The user sits in front of their laptop and watches a short sequence of emotionally provocative images. While this happens, the webcam captures live video with the user's consent and feeds it to a facial emotion recognition model (DeepFace), which detects how the user's face reacts in real time and classifies expressions into emotional states.

The system then asks the user a few questions, which they answer aloud. Their voice is transcribed by Whisper (faster-whisper), running locally. The resulting text is passed to a sentiment analysis model, which identifies the emotional tone behind what the user said.

All the data collected so far - facial reactions, transcribed answers, sentiment scores - is then fed to a local language model running through Ollama. The model synthesises everything it has observed and opens an interactive conversation with the user, letting them ask questions and explore their results.

Finally the system delivers a personalised psychological profile consisting of a summary of detected emotional patterns, reported concerns, and personalised wellbeing recommendations, alongside practical suggestions: books to read, meditation practices, focus areas to work on, and lifestyle suggestions. If the profile indicates serious concerns, the system actively encourages the user to seek professional help from a qualified therapist.

4. Implementation

Orchestration overview

A `pipeline.py` module acts as the orchestrator. Each model lives in its own module with a clean interface (a class or a run function returning a small structured result), and `pipeline.py` is solely responsible for sequencing the stages and passing data between them. This keeps each model self-contained and independently testable, and allows stages to be added without changing the modules already written.

All four stages now run end to end within a single `run_pipeline()` function, wrapped in the guided session flow described later in this chapter: facial emotion detection (vision) produces a per-scene emotion summary; speech-to-text (audio) transcribes a recorded response; text-emotion analysis (text) classifies that transcript; and a local language model (Ollama) synthesises all three outputs into a written psychological reflection. Each module uses a try/except import pattern (importing as part of the `src` package when run through `pipeline.py`, or as a standalone module when run directly for isolated testing), which is what allowed every stage to be built and unit-tested independently before being wired in.

Every run is also logged: once the follow-up chat ends, `save_session_log()` writes a single timestamped JSON file containing every stage's output (the emotion summary, the three question-and-answer pairs, text-emotion result, and full profile) alongside the complete chat transcript, together with whether consent was given and whether the run was a demo. Recording consent and demo status in the log matters for the user study specifically: it distinguishes a real participant session from a demonstration run, and evidences that consent was obtained per session rather than assumed. This turns each session into structured, reusable data rather than terminal output that has to be copied by hand, which matters both for the results reported in this Evaluation chapter and for the planned six-to-ten person user study, where session data will need to be reviewed and compared systematically rather than one run at a time.

Stage 1: Facial emotion recognition (DeepFace)

DeepFace classifies emotion from an image using a convolutional neural network (CNN). A webcam frame is first passed to a face detector that crops the face. The cropped face goes through convolutional layers, which slide filters over the image to pick out patterns (edges, then shapes, then features like a smile or a raised brow), and pooling layers, which shrink the data to keep only the strongest signals. A final softmax step turns the network's scores into probabilities across seven emotions and returns the highest one. Each frame is analysed on its own with no memory of previous frames, which is why neutral or ambiguous faces are often misread.

The vision prototype was refactored from a flat script into an importable `run_session()` function returning a timeline of (time, scene, emotion) readings, plus a pure `summarise_timeline()` helper, kept separate so it can be unit-tested without hardware.

Stage 2: Speech-to-text (Whisper / faster-whisper)

Whisper is a transformer model that converts audio to text. The audio is first turned into a mel spectrogram, a picture of how its frequencies change over time. An encoder compresses this into internal features, and a decoder then produces the transcript one token at a time, predicting each next word from the audio and the words so far. Because it models language and not just sounds, it handles continuous speech and punctuation well.

The project uses faster-whisper rather than the original openai-whisper, since it runs roughly four times faster on CPU at equivalent accuracy and supports int8 quantisation to reduce memory use - both matter given the project runs locally with no GPU. The component is built as a Transcriber class: the model is loaded once in `__init__` and reused for every subsequent call, avoiding the cost of reloading it each time.

Live microphone input was added via a new recorder.py module, wrapping sounddevice. Rather than recording for a fixed duration, `record_audio()` streams audio into memory on a background callback until the user presses Enter, then writes it to a 16kHz mono WAV file - the sample rate Whisper expects internally, avoiding an extra resampling step. `pipeline.py` now records live by default, taking a separate recording for each of the three spoken questions rather than one continuous capture, so each answer can be attributed to the question that prompted it. A `--demo` flag switches back to the bundled `sample.wav`, which is kept as a fallback for offline demonstrations (e.g. the Topic 9 peer review) or if a working microphone is unavailable.

Stage 3: Sentiment / text-emotion analysis

The text stage is implemented in `sentiment.py`, following the same structural pattern as the Transcriber class: a `SentimentAnalyzer` class loads the chosen model once at construction and exposes a single `analyze(text)` method that returns a small structured result (an `EmotionResult` namedtuple of label and confidence score), so the rest of the pipeline always receives a predictable shape regardless of which model sits underneath.

`j-hartmann/emotion-english-distilroberta-base` was selected after benchmarking it against VADER and `tabularisai/multilingual-sentiment-analysis` on a fixed set of test transcripts (see Evaluation, Stage 3 results, for the full comparison). Its key advantage is sharing DeepFace's seven-emotion label space, so facial and verbal emotion can later be compared directly by the language model rather than reconciled across two different vocabularies.

As with the audio module, the transformers import sits inside `__init__` rather than at the top of the file, so `sentiment.py` can be imported by the pipeline without the (fairly heavy) transformers/torch dependency being installed, unless a `SentimentAnalyzer` is actually constructed. The `analyze()` method also short-circuits on empty or whitespace-only input, returning a neutral, zero-confidence result rather than calling the model - an empty transcript is a valid pipeline state (e.g. a dropped audio segment or a silent response), not an error condition.

The stage was first verified in isolation, then integrated into `pipeline.py` directly after the transcription stage. It now receives the combined question-and-answer transcript

assembled by `session.format_qa_transcript()` rather than a single recording's text, so the emotional tone it scores is that of the user's three spoken answers taken together. A full pipeline run on a sample recording ("This is a recorded message to test my program and to make sure that it will also work on a live feed from the microphone. Thank you.") correctly returned neutral (0.862) for this factual, unemotional sentence, confirming the stage integrates cleanly and the transcript passes to it in a usable form.

Stage 4: Local language model (Ollama)

The language stage is implemented in `profile_generator.py` as a `ProfileGenerator` class, following the same lazy-import pattern as the other three modules (the ollama package is only imported inside `__init__`, so the module can be imported elsewhere in the pipeline without it installed). The class holds a running conversation history on the instance, exposing both a `generate_profile()` method for the initial reflection and a `chat(user_message)` method for follow-up turns that share the same context, so the caller does not need to manage message state separately.

A `run_chat_loop()` function was then added to `pipeline.py` to close this gap: after the initial profile prints, the pipeline takes user input via `input()`, passes each message to `generator.chat()`, and prints the reply, repeating until the user types a quit word (quit/exit/q/bye/stop) or sends an interrupt. Because it reuses the same `ProfileGenerator` instance, the model retains full session context - the facial emotion summary, transcript, and profile it just wrote - for every follow-up turn, rather than starting a fresh, context-free conversation.

A single structured prompt is assembled from all three upstream stages: the per-scene dominant facial emotions from Stage 1, the transcript from Stage 2, and the transcript's classified emotional tone from Stage 3. The model is instructed to produce (1) a summary of the emotional patterns observed, explicitly including any mismatch between facial expression and spoken tone, (2) one or two focus areas, and (3) two or three general wellbeing recommendations.

A fixed system prompt is set once at construction and sent with every request. It explicitly forbids diagnostic or clinical language, instructs the model to stay warm and non-clinical, and requires it to recommend professional help when the observed patterns suggest significant or persistent distress. This is a direct implementation of the severity-assessment design decision (see Design chapter) and of the ethical requirement, established through the literature review (Ezell, 2026; Riedl et al., 2024), that the system must never attempt to replace a qualified professional.

A secondary, non-authoritative safety net was also added: `generate_profile()` checks the model's reply for keywords such as "professional" and "therapist" and sets a `seek_help` boolean flag on the returned Profile. This is not relied upon to make the system safe - the system prompt is the primary control - but gives the calling code (e.g. a future UI layer) a simple, cheap way to know whether to surface a resources link.

llama3.2 (the smallest available Ollama model, approximately 2GB) was used for this integration test, on the basis that it requires no dedicated GPU and was already confirmed to run acceptably on the target hardware during setup.

llama3.2 was compared against qwen2.5:3b on an identical sample prompt (see Evaluation, Table 3 for full numbers). qwen2.5:3b was faster on both time-to-first-token and total generation time, and produced a more concise reply, but it did not recommend professional help for a prompt containing a fear signal similar to earlier llama3.2 tests that did trigger that recommendation. llama3.2 was kept as the primary model on this basis: for a system whose core design requirement is severity assessment, a model that reliably surfaces the seek-help recommendation is prioritised over one that is faster but less consistent on this specific behaviour.

Integration test

With all four modules wired into pipeline.py, a full run was carried out: Stage 1 captured a (simulated) per-scene emotion summary, Stage 2 transcribed a short test recording, Stage 3 classified its tone as neutral, and Stage 4 received all three outputs and generated a full profile. The model correctly identified a mismatch between the fear reading recorded during the "sad scene" and the otherwise neutral tone of the transcript, and, per its system prompt, closed with an explicit recommendation to seek professional support, correctly setting the seek_help flag to True:

"...I do want to acknowledge the fear that showed up during your sad scene. If you're struggling to manage overwhelming emotions or if this feeling persists, it's essential to reach out to a mental health professional for support."

This confirms the full four-stage pipeline runs without errors and that the severity-assessment behaviour required by the system's design works as intended, at least in this initial test. The per-scene emotion data used in this run was still the simulated placeholder timeline described in Stage 1, not a real user session, so the result demonstrates correct integration rather than a validated real-world profile.

A second, fully live end-to-end run was then carried out with the mic-recording and interactive-chat additions in place: a real spoken response was recorded via recorder.py, transcribed, classified, and used to generate a profile, after which several follow-up questions were exchanged through run_chat_loop(). This confirmed that live microphone input, the full four-stage pipeline, and the interactive chat all work together correctly. See Evaluation, Stage 4 results, for a limitation identified during this test.

Session flow and user interaction

The four model stages are wrapped in a guided session implemented in session.py, so that what a user encounters is a structured, consent-first experience rather than the developer pipeline the stages alone provide. The flow runs in seven steps: consent and disclaimer, the stimulus sequence with live facial analysis, three spoken questions, processing, the generated profile, the follow-up conversation, and a closing message pointing to professional support.

Consent is handled by `get_consent()`, which prints a disclaimer stating what will be captured (webcam and microphone), what the system will not do (diagnose, or transmit anything off the machine), that the session is written to a local log file, and how to stop at any point. The check fails closed: only an explicit "yes" proceeds, while a blank line, a refusal, an end-of-file or a keyboard interrupt are all treated as consent withheld. The end-of-file case is the one worth naming, since a non-interactive run with no available stdin would otherwise raise an exception rather than declining cleanly, and an unclear answer must never be read as agreement. Where consent is not given, `run_pipeline()` returns before any stage executes, so nothing is captured, recorded or logged for a declined session.

The three spoken questions ("How are you feeling right now?", "What did those images bring to mind?", "Is anything weighing on you?") are held in a module-level constant and asked one at a time by `run_spoken_questions()`, with each answer recorded and transcribed separately rather than as a single continuous recording. `format_qa_transcript()` then combines them into the one string the text-emotion and language stages consume, keeping each question visible alongside its answer so the language model can see what each response was addressing rather than receiving three unlabelled sentences run together.

The session closes with `print_closing()`, which always displays signposting to professional support and adds a more direct prompt when the `seek_help` flag is set. Showing the resources on every session rather than only flagged ones is deliberate: it implements the Design chapter's argument that this tool is a first step toward care-seeking, which would be undermined if the signpost appeared only on the sessions a small local model happened to classify as concerning.

A terminal interface plus the existing OpenCV window was judged sufficient for this prototype. The interaction-design questions that actually matter for a first-step self-reflection tool - whether consent is informed and freely given, whether the questions invite genuine reflection, and whether the closing message routes the user somewhere real - are all answerable in a terminal, and a graphical interface would have consumed build time without addressing any of them. This is a scope decision rather than an oversight, and is revisited in the Conclusion as candidate further work.

--demo mode auto-confirms consent and reuses the bundled `sample.wav` for all three questions, so that a peer reviewer can run a complete session without a webcam, microphone or Ollama installation. The reuse is a documented limitation of demo mode rather than a defect: only one sample recording ships with the project, and the mode prints this to the terminal at each question so that three identical answers are not mistaken for three real ones.

Visual representation of the product / results

5. Evaluation

Testing methodology

Since this project uses pre-trained models rather than training one from scratch, evaluation focuses less on large user studies and more on whether the models run well on the target hardware and work together as one pipeline. Testing is split into three stages: checking each model in isolation, testing how the stages connect, and getting feedback from real users.

The first stage checks each model on its own. For every model, minimum hardware requirements were checked against the documentation, the model was installed locally and run on sample inputs to confirm usable output, and processing time and memory use were recorded. If a model proved too slow or too heavy, it was swapped for a lighter alternative (e.g. Vosk instead of Whisper, or VADER instead of a transformer-based sentiment model) - this swap is the main fallback built into the design.

The second stage tests how the models connect: whether output from one stage reaches the next in a usable form (e.g. Whisper's transcription passing cleanly to the sentiment model), watching for mismatched data formats, empty or dropped inputs, and total pipeline processing time.

The third stage tests the experience with real users. Between six and ten testers rate the result with a short Likert-scale survey (1-5) on accuracy, clarity, and usefulness, alongside a pre-session self-report used to check how well the generated profile matches the user's own account.

Stage 1 results: Facial emotion recognition

The model loaded and ran successfully on the test machine (Intel Core i7-1185G7, 16GB RAM, no dedicated GPU), confirming local execution is viable without specialist hardware. With FPS and analysis-latency instrumentation added, the system achieved approximately 127 FPS for video capture and 73ms per emotion analysis. Since analysis runs only every 10th frame, the display remained fluid while CPU load stayed manageable, confirming the pipeline is fast enough for real-time use on consumer hardware.

Across a 20-second test cycling through four labelled scenes, the model correctly identified the dominant emotion matching the deliberately expressed emotion in all four cases (surprise, happy, sad, neutral), confirming the emotion-to-scene tagging works as intended.

A clear limitation emerged in the emotion timeline: between deliberate expressions, when the face was relatively neutral, the model often returned fear or disgust instead of neutral. This suggests the model struggles with resting or ambiguous faces and tends to over-detect negative emotions with no strong signal present - a known limitation of frame-by-frame analysis with no temporal context.

A subsequent full-session run through the completed guided flow produced a sharper version of the same problem. Across all four scenes, every reading returned the same dominant emotion (sad: 10, 15, 15 and 12 readings for the threatening, joyful, sad and relaxing scenes respectively), with no variation between scenes that were designed to provoke markedly different responses. Unlike the earlier test, this run involved no deliberately expressed emotions, so it represents the resting-face condition specifically, and it indicates that the model's behaviour on an unperforming face is not merely noisy but can be uniformly wrong in a single direction. This has a direct consequence for the system as a whole: the language stage receives a per-scene summary it treats as meaningful, and cannot distinguish a genuine emotional pattern from a constant misclassification, so a uniformly incorrect vision stage propagates silently into a confident-sounding profile.

Stage 2 results: Speech-to-text

The audio stage was verified as part of the full pipeline integration run: a sample recording was transcribed correctly and fluently ("This is a recorded message to test my program and to make sure that it will also work on a live feed from the microphone. Thank you."), confirming faster-whisper produces clean, directly usable text for the downstream sentiment stage.

faster-whisper was then benchmarked in isolation against the original openai-whisper on the same audio file, both at the tiny model size (see Table 1a). faster-whisper was faster on transcription (0.54s vs 1.05s), used substantially less peak memory (13.4MB vs 72.1MB), and was more accurate on this file: openai-whisper again mis-transcribed "microphone" as "Micah Cup", while faster-whisper transcribed the sentence correctly in full. openai-whisper additionally required a system-level ffmpeg installation to run at all, which faster-whisper does not - a further practical point in favour of the chosen library for a tool intended to be simple to set up locally.

Model	Model load time	Transcription time	Peak memory	Notes
faster-whisper (tiny)	0.74s	0.54s	13.4 MB	Transcribed the test sentence correctly in full
openai-whisper (tiny)	0.44s	1.05s	72.1 MB	Mis-transcribed one word ("microphone" -> "Micah Cup"); required a separate system-level ffmpeg install to run

Table 1a. faster-whisper vs openai-whisper on an identical audio file, tiny model size, on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU. Peak memory measured with

tracemalloc, isolated per model; this captures Python-level allocations only and likely understates true peak memory, since it does not see inside PyTorch's C/C++ tensor backend.

Stage 3 results: Sentiment / text-emotion analysis

Three candidate models were benchmarked on the same 10 test transcripts (see `test_transcripts.py`): VADER, a rule-based sentiment analyser (Hutto & Gilbert, 2014); j-hartmann/emotion-english-distilroberta-base, a transformer trained on the same seven-emotion label space as DeepFace; and tabularisai/multilingual-sentiment-analysis, a 5-class sentiment model.

j-hartmann was selected. It correctly labelled 6 of 10 transcripts, including the surprise and disgust cases that both other models missed entirely, and its shared label space with DeepFace (anger, disgust, fear, joy, neutral, sadness, surprise) means facial and verbal emotion can be compared directly downstream (e.g. "your face showed sadness while your words expressed joy"). At 44ms per input it remains far faster than needed for a system driven by user speech, where the bottleneck is transcription and conversation, not sentiment scoring.

Table 1 summarises average performance per candidate model across all 10 test transcripts.

Model	Avg latency (ms/input)	Memory	Notes
VADER	0.18	Negligible (rule-based, no model weights)	Fastest by far, but only 3-class (positive/negative/neutral); cannot detect surprise or disgust and missed the mixed-emotion case entirely
j-hartmann/ emotion-english- distilroberta-base	44.31	See note below table	6/10 correct; shares DeepFace's 7-emotion label space; only model to correctly catch surprise and disgust
tabularisai/ multilingual- sentiment-analysis	30.87	See note below table	Heavily biased toward "Negative" - 8/10 predictions were Negative, including clearly positive/neutral cases

Table 1. Average latency per candidate model on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU. Memory: tracemalloc reported 193.2MB peak Python memory across the full benchmark run (all three models); per-model isolation was not measured in this run and is noted as a limitation.

Table 2 shows the label each model predicted for every test transcript, against the expected label. Bold indicates a mismatch with the expected label.

Text	Expected	VADER	j-hartmann	tabularisai
Honestly I feel really good today...	happy	positive	joy	Positive
I don't know, I've just been feeling really down...	sad	negative	joy (wrong)	Negative
It was a normal day, nothing much happened.	neutral	neutral	neutral	Neutral
I keep worrying about this presentation...	fear	negative	fear	Negative
It's just so frustrating, nobody ever listens...	anger	negative	sadness (wrong)	Negative
I'm excited about the new job but also kind of scared...	mixed	negative	fear	Negative
I had this weird dream last night...	fear	negative	fear	Negative
Wait, I did not expect that at all, that's wild.	surprise	neutral	surprise	Negative
That really put me off, I couldn't stand it.	disgust	neutral	disgust	Negative
I'm fine.	neutral	positive	neutral	Positive

Table 2. Predicted label per model against the expected label for each test transcript, run on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU.

Stage 4 results: Local language model (Ollama)

llama3.2 was integrated and tested as part of the full four-stage pipeline run described in the Implementation chapter. The model produced a coherent, non-diagnostic reflection that correctly identified an emotional mismatch across the facial and text-emotion signals (fear detected in the "sad scene" against an otherwise neutral spoken tone), offered three concrete, general wellbeing recommendations (journaling, mindful breathing, time outdoors), and correctly triggered the seek_help flag by recommending professional support given the distress signal - confirming the severity-assessment design requirement is functioning as intended for at least this test case.

This result is evidence of correct integration, not of overall output quality or safety across a range of inputs: it is a single run on simulated per-scene emotion data (see Stage 1), not real user sessions, and only one model has been tried so far.

The interactive follow-up conversation described in the Design chapter is now also functional: run_chat_loop() takes user input after the profile is printed and passes it to ProfileGenerator.chat(), which retains the full session context (facial emotion summary, transcript, and the profile itself) across turns. This was verified manually across several follow-up turns, with the model's replies generally remaining consistent with, and referencing, the profile it had just generated.

One notable limitation surfaced during this testing: on at least one follow-up turn, llama3.2 produced a reply that misjudged the intent of an ambiguous user message and responded with an inappropriate, unrelated safety refusal rather than asking for clarification or responding sensibly. The reply was incorrect and out of context rather than unsafe in itself, but it illustrates a real risk for this project: a small local model can misread short or ambiguous input in unpredictable ways, and for a tool operating in a mental-health-adjacent context, an erratic or nonsensical response at the wrong moment could undermine user trust or feel dismissive. This is treated as an open risk to test more systematically (varied phrasing, ambiguous input, edge cases) rather than something resolved by this build, and is a strong candidate topic for the Design chapter's discussion of model limitations and for the final report's evaluation of alternative models.

llama3.2 and qwen2.5:3b were compared on an identical sample prompt built from a simulated emotion summary, transcript, and text-emotion result (see ollama_benchmark.py). Results are given in Table 3.

Model	Time to first token	Total generation time	Approx. size	Output quality (notes)
llama3.2:3b	10.46s	28.04s	~2GB	1332-character reply; explicitly recommended

Model	Time to first token	Total generation time	Approx. size	Output quality (notes)
qwen2.5:3b	8.79s	18.99s	~2GB	professional support given the fear signal 789-character reply, more concise; covered the same emotional patterns but did not recommend professional support for a comparable distress signal

Table 3. Local LLM comparison for profile generation, run on an identical sample emotion summary (facial emotions + transcript + sentiment scores) on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU.

qwen2.5:3b is roughly 30-40% faster on both metrics and produces noticeably shorter output. However, the missing seek-help recommendation on a prompt with a comparable distress signal to earlier llama3.2 tests is a meaningful finding given the project's severity-assessment design requirement (see Design chapter): a faster model is of limited value if it is less consistent at surfacing this specific, safety-relevant behaviour. llama3.2 was therefore retained as the primary model for the current build. This is based on a single comparison prompt rather than a systematic test and should be treated as an initial finding, not a conclusive result - see the structured chat-testing item above.

Session logging

Session logging (save_session_log()) was verified with a full live run: webcam capture, a spoken response, sentiment classification, profile generation, and a short follow-up chat, all written to a single timestamped JSON file. It was re-verified after the guided session flow was added, confirming that the consent status, demo-mode flag, and all three question-and-answer pairs are captured alongside the existing stage outputs. The chat history in the log confirmed the model correctly retained context across turns, referencing the earlier profile discussion when later asked what it remembered.

This test run also surfaced another example of the model over-interpreting low-signal input, in addition to the finding noted above: given a microphone test phrase ("1, 2, 3, test") with neutral sentiment, the model constructed a confident, detailed narrative about the user's emotional state rather than recognising the input as a content-free test utterance. As with the earlier finding, this is not a crash or a safety failure, but it further supports the

case for more systematic testing of how the language stage handles low-information or ambiguous input before this system could be relied upon with real users.

Unit testing

A pytest suite covers the deterministic, hardware-independent logic across all four testable modules, using mocks so tests run offline and instantly rather than loading real models, opening a microphone, or contacting a real Ollama server. For the vision module, `get_current_scene()` and `summarise_timeline()` are tested against fixed inputs and expected outputs including edge cases; live webcam capture is not unit-tested, since its input can't be reproduced deterministically, and is instead verified through feature testing.

For `sentiment.py`, the transformers pipeline is mocked so the tests check `SentimentAnalyzer`'s own logic rather than the model's classification accuracy (that is instead covered by `sentiment_benchmark.py` against real model output): the empty/whitespace-input guard returns neutral without calling the model, the top label and score are extracted correctly regardless of the exact output nesting shape returned, and the model is confirmed to load only once across multiple `analyze()` calls rather than being reloaded each time.

For `profile_generator.py`, the ollama package is similarly mocked to isolate `ProfileGenerator`'s own logic from the language model's output quality: the system prompt is confirmed to be set as the first history entry, `_build_prompt()` is confirmed to include all three upstream stages' outputs (and to handle an empty summary or transcript without raising), the `seek_help` keyword check is confirmed to trigger correctly (including case-insensitively) and to correctly stay False when no such keyword appears, and `chat()` is confirmed to both return the model's reply and share conversation history with an earlier `generate_profile()` call in the same session.

For `session.py`, both the microphone and the consent prompt are mocked, isolating the session logic from hardware and from a human typing at a terminal. The consent gate is tested across its full range of answers - an explicit "yes" (including with surrounding whitespace and mixed case), an explicit refusal, a blank line, an end-of-file and a keyboard interrupt - confirming that only the first proceeds and that the remainder fail closed rather than raising. The question stage is confirmed to ask all three questions in the defined order and to take a separate recording for each in live mode, while demo mode is confirmed to reuse the single bundled sample for all three. Transcript assembly is checked for content and ordering, including the empty case, and the closing message is confirmed to display professional-support signposting on every session while adding the stronger prompt only when the `seek_help` flag is set.

The full suite (36 tests across all four modules) passes: 7 vision tests, 6 sentiment tests, 8 profile-generator tests, and 15 session-flow tests, running in seconds with no external services, hardware or downloaded models required.

A `pytest.ini` file (`pythonpath = src`) was added at the project root so test files import directly from `src` (e.g. from `sentiment` import `SentimentAnalyzer`) without a manual `sys.path` workaround in every test file, and a matching `.vscode/settings.json` entry lets the editor's

static analysis resolve the same imports, removing false-positive unresolved-import warnings during development.

Adding the fourth test module surfaced a latent inconsistency in this arrangement worth recording, since it is a genuine testing-infrastructure finding rather than a coding error. Two import styles had accumulated across the test suite: the vision tests import the package (from `src import emotion_detector`), which requires the project root on the path, while the remaining modules import flat module names, which requires `src` itself on the path. `pytest.ini` satisfied only the second, so the first worked only when `pytest` happened to be invoked in a way that placed the project root on `sys.path` incidentally. Running the full suite in one command exposed this. A root-level `confest.py` that adds the project root to `sys.path` resolves it for every invocation, making the suite's behaviour independent of how it is launched.

Figure. The editor's static analyser (Pylance) flagging an unresolved import before `pytest.ini` and `.vscode/settings.json` were added. The import worked correctly at runtime throughout (the test suite passed regardless) - this was purely an editor-level false positive caused by the analyser not knowing about the `src/` path used at test time, and was resolved by the configuration described above.

Peer review results

User testing

Inclusive design and demographic limitations

Overall assessment so far

The project is on track against its stated goal: a locally-run system combining multiple pre-trained models into a coherent psychological profile. All four stages are now implemented, integrated, and confirmed working together in a fully live end-to-end run, and are wrapped in the complete guided session flow: consent, stimulus with facial emotion detection, three separately recorded spoken questions, transcription, text-emotion classification, profile generation, an interactive follow-up conversation, and a closing message signposting professional support. Vision has been formally benchmarked for capture speed and scene-tagging accuracy, with one documented limitation around neutral-face misclassification. Text-emotion analysis, speech-to-text and language-model generation have each in addition been benchmarked against named alternatives and justified on the evidence (Tables 1, 1a, 2 and 3). Vision is now the only stage whose chosen model has not been compared against an alternative.

The main work remaining before the final report is not building new stages, but strengthening what exists: replacing the simulated per-scene stimulus with real calibrated content, benchmarking DeepFace against at least one alternative facial-emotion model - now the highest priority of these, given the resting-face result reported above - more systematically testing the chat stage for erratic or off-topic replies across both candidate language models (an initial limitation was identified during live testing with `llama3.2` - see

Evaluation, Stage 4 results), and running the planned six-to-ten person user study. The pipeline's architecture (small, independently-testable modules coordinated by pipeline.py) means each of these is an isolated change rather than a restructuring effort.

6. Conclusion

References